ZPRACOVÁNÍ SOUBORŮ, KÓDOVÁNÍ A UNICODE, ZPRACOVÁNÍ TEXTU, REGULÁRNÍ VÝRAZY

13.10.2026

Luboš Šmídl 2026

Zpracování souborů

Práce se soubory je běžnou součástí programování – umožňuje data uložit na disk nebo je naopak načíst. Python k tomu poskytuje vestavěnou funkci `open()`.

```
open(jméno_souboru, režim='r', encoding=None, ...)
```

Režimy pro textové soubory:

- `'r'` – **read** (čtení), výchozí režim. Soubor musí existovat, jinak `FileNotFoundError`.
- `'w'` – **write** (zápis). Existující obsah se **smaže hned při otevření**! Neexistující soubor se vytvoří.
- `'a'` – **append** (připojení na konec). Neexistující soubor se vytvoří.
- `'x'` – **exclusive creation**. Vytvoří nový soubor; pokud už existuje, vyvolá `FileExistsError`.

Binární soubory: k režimu přidáme `'b'`, např. `'rb'`, `'wb'`.

Textový režim: při čtení **dekóduje** bajty na řetězec `str`, při zápisu **kóduje** `str` na bajty podle `encoding`; navíc převádí konce řádků. **Binární režim:** žádná konverze, pracujeme s typem `bytes`.

Otázka: Co se stane, když existující soubor s výsledky měření omylem otevřete v režimu `'w'`?

Tři strategie čtení

- **Celý soubor najednou –** `f.read()`: pohodlné, ale jen pro soubory, které se vejdou do paměti
- **Po řádcích –** `for radek in f:` – textové soubory libovolné velikosti
- **Po blocích –** `f.read(n)` – binární soubory a velká data

```
fr = open('vstup.txt', 'r', encoding='utf-8')
```

- U textových souborů **musíme znát kódování** a uvést ho parametrem `encoding`
- Bez `encoding` použije Python výchozí kódování systému:
  - Windows s češtinou typicky **cp1250**, Linux/macOS **UTF-8**
  - ⇒ stejný program načte stejný soubor na různých počítačích různě
- Pravidlo: **`encoding` uvádějte vždy** (podrobně v části Kódování)

Binární soubory


```
BUF_SIZE = 1024  # 1 KiB

with open('vstup.bin', 'rb') as fr, \
     open('vystup.bin', 'wb') as fw:
    celkem_bajtu = 0
    while True:
        data = fr.read(BUF_SIZE)
        if not data:           # b'' = konec souboru
            break
        celkem_bajtu += len(data)
        fw.write(data)

print('Zapsáno', celkem_bajtu, 'bajtů')
```

V praxi totéž jedním řádkem: `shutil.copyfileobj(fr, fw)`

Binární soubory – poznámky

- Binární data otevíráme **vždy** s `'b'` – textový režim by bajty dekódoval a převáděl konce řádků → poškozená data nebo `UnicodeDecodeError` (na každém OS)
- Proč po blocích? Soubor může být **větší než paměť**, nebo jde o **proud dat** (síť, roura) bez známé délky
- `.read(n)`:
  - uprostřed souboru vrátí `n` bajtů
  - na konci může vrátit méně než `n` bajtů
  - po dosažení konce vrací prázdné `b''` (při každém dalším volání znovu) – **žádná výjimka**
- Po skončení práce je nutné soubory **zavřít**:
  - explicitně metodou `close()`
  - automaticky v bloku `with` (doporučeno)

Příkaz with


```
with open('vstup.bin', 'rb') as fr, \
     open('vystup.bin', 'wb') as fw:
    ...   # práce se soubory
```

- Na začátku se otevře `'vstup.bin'` a objekt souboru se uloží do `fr`
- Hned poté se otevře `'vystup.bin'` a uloží se do `fw`
- V těle příkazu pracujeme se soubory (čtení, zápis)
- Po skončení těla se **oba soubory automaticky zavřou** – i když dojde k výjimce
  - pokud `'vystup.bin'` nepůjde otevřít, již otevřený `'vstup.bin'` se korektně zavře
- Zpětné lomítko `\` na konci řádku: příkaz pokračuje na dalším řádku
- Od Pythonu 3.10 lze místo `\` použít závorky:

```
with (open('a.txt', encoding='utf-8') as fa,
      open('b.txt', 'w', encoding='utf-8') as fb):
```

Textové soubory

Načtení všech řádků najednou a jejich zápis:

```
with open('vstup.txt', 'r', encoding='utf-8') as fr, \
     open('vystup.txt', 'w', encoding='utf-8') as fw:
    lines = fr.readlines()
    fw.writelines(lines)
print('Zpracováno', len(lines), 'řádků')
```

Čtení:

- `fr.read()` – celý obsah jako jeden řetězec
- `fr.readline()` – jeden řádek
- `fr.readlines()` – seznam všech řádků (každý i s `'\n'`); celý soubor je v paměti

Zápis:

- `fw.write(s)` – zapíše řetězec, nový řádek **nepřidá**
- `fw.writelines(seznam)` – zapíše řetězce za sebou, **nic mezi ně nevkládá**

Čtení po řádcích


```
with open('data.txt', 'r', encoding='utf-8') as fr, \
     open('kopie.txt', 'w', encoding='utf-8') as fw:
    pocet_radku = 0
    for radek in fr:          # iterujeme po řádcích
        pocet_radku += 1
        fw.write(radek)       # radek už obsahuje '\n'

print(f"Zpracováno {pocet_radku} řádků.")
```

- Soubor je **iterovatelný** – cyklus `for` vrací postupně jednotlivé řádky
- Celý soubor se **nenačítá do paměti** – funguje i pro velmi velké soubory
- Na konci souboru cyklus sám skončí

Textové soubory – zásady

- U textových souborů vždy uvádějte **jméno souboru** a **kódování**
- Cyklus `for` přes soubor vrací řádky; na konci souboru skončí
- Proměnná `radek` **končí znakem nového řádku** `'\n'` (kromě posledního řádku, pokud soubor nekončí `'\n'`)
- Metoda `.write()` nový řádek **nevkládá**
- Funkce `print()` nový řádek **vkládá** – pozor na zdvojení:

```
print(radek, file=fw)                 # chyba: \n dvakrát
print(radek, end='', file=fw)         # správně
print(radek.rstrip('\n'), file=fw)    # správně
```

- Odstranění konce řádku: `radek.rstrip('\n')`, případně `radek.strip()` (odstraní i mezery na okrajích)

Kódování textu a Unicode

Unicode je mezinárodní standard, který definuje **jednotnou sadu znaků** pro (téměř) všechny jazyky světa a množství symbolů.

Každému znaku (písmenu, číslici, symbolu, emoji…) je přiřazeno **unikátní číslo** – tzv. **kódový bod** (code point).

Příklady: „A“ = U+0041, „Ž“ = U+017D, sněhulák ☃ = U+2603

```
>>> ord('Ž'), hex(ord('Ž')), chr(0x2603)
(381, '0x17d', '☃')
```

Unicode tedy můžeme chápat jako **abstraktní množinu znaků** a přiřazení čísel těmto znakům. Neříká, jak čísla uložit do bajtů.

UTF-8, UTF-16, UTF-32

Jak kódové body uložit do bajtů, určuje **znakové kódování** (encoding) – pravidlo převodu čísla znaku na posloupnost bajtů. Nejpoužívanější kódování Unicode:

- **UTF-8** – proměnná délka: 1 bajt pro ASCII (základní latinka, číslice, základní interpunkce), 2–4 bajty pro ostatní znaky
- **UTF-16** – 2 bajty pro většinu běžných znaků, méně časté znaky 4 bajty
- **UTF-32** – vždy 4 bajty na znak

```
>>> 'Ž'.encode('utf-8')
b'\xc5\xbd'                          # 2 bajty
>>> len('Ž'.encode('utf-32-le'))
4
```

Unicode není kódování. Unicode definuje mapu znaků, kódování (UTF-8, UTF-16, …) definují, jak bude tato mapa reprezentována v bajtech.

Kódování před Unicode

Před Unicode existovala řada **národních kódování** (kódových stránek):

- **ASCII** – 7bitový standard: 128 znaků (anglická abeceda, číslice, základní symboly); hodnoty 128–255 využívají 8bitové kódové stránky, každá jinak
- **ISO 8859-2 (Latin-2)** – středoevropské jazyky (čeština, polština…), jednobajtové, 256 hodnot
- **Windows-1250 (cp1250)** – středoevropská kódová stránka Windows; podobná Latin-2, ale některé znaky jsou jinde:

```
>>> 'š'.encode('cp1250'), 'š'.encode('iso-8859-2')
(b'\x9a', b'\xb9')
```

- obdobně **cp1251** (azbuka), **cp1252** (západní Evropa), **KOI8-R**, **Shift-JIS** a stovky dalších

Problém: jednobajtové kódování pojme jen 256 znaků → nelze v něm zapsat např. češtinu a ruštinu zároveň; stejný bajt znamená v různých kódováních různé znaky. Unicode řeší obojí jednou tabulkou pro všechny.

Řetězce a bajty

- `str` – posloupnost **znaků** (kódových bodů); s tímto typem pracujeme v programu
- `bytes` – posloupnost **bajtů** (0–255); to je v souboru, na disku, v síti
- **Převod:** `str.encode(kódování)` → `bytes`, `bytes.decode(kódování)` → `str`

```
>>> s = 'Příliš'
>>> len(s), len(s.encode('utf-8'))
(6, 9)
```

Mojibake – bajty dekódované jiným kódováním, než ve kterém byly zapsány:

```
>>> 'Žluťoučký'.encode('utf-8').decode('cp1250')
'Ĺ˝luĹĄouÄŤkÄ˝'
>>> 'Žluť'.encode('cp1250').decode('utf-8')
UnicodeDecodeError: ... invalid start byte
```

Otázka: Editor zobrazuje text jako „Ĺ˝luĹĄouÄŤkÄ˝“. V jakém kódování byl soubor uložen a v jakém ho editor čte?

Kódování – praktické rady

- **Preferujte UTF-8** – dnešní de facto standard (web, Linux, Python)
- **Vždy uvádějte `encoding`** u funkce `open()`
- **Zdrojové kódy** Pythonu ukládejte v UTF-8 – Python 3 je tak čte ve výchozím stavu (PEP 3120); deklarace `# -*- coding: utf-8 -*-` už není nutná
- **Neznámé kódování:** knihovny `chardet` / `charset-normalizer` ho umí odhadnout – ale **jen odhadnout**; lepší je znát ho z kontextu
- **Chyby dekódování** ošetřete:

```
try:
    with open(cesta, encoding='utf-8') as f:
        text = f.read()
except UnicodeDecodeError:
    with open(cesta, encoding='cp1250') as f:
        text = f.read()
```

- Parametr `errors=`: `'strict'` (výchozí – výjimka), `'replace'` (nahradí znakem □), `'ignore'` (zahodí – ztráta dat!)
- CSV pro Excel: `encoding='utf-8-sig'` (UTF-8 se značkou BOM)

Text z webu = soubor po síti

- Webová stránka je pro program jen **posloupnost bajtů** poslaná po síti
- Kódování oznamuje server v hlavičce `Content-Type`, případně stránka v `<meta charset="UTF-8">`

```
import requests
url = 'https://www.kky.zcu.cz/cs/Department/People.html'
r = requests.get(url, timeout=10)
print(r.headers['Content-Type'])  # text/html;charset=UTF-8
print(r.content[:15])   # bytes – přesně to, co přišlo po síti
print(r.text[:15])      # str – dekódováno podle r.encoding
with open('people.html', 'w', encoding='utf-8') as f:
    f.write(r.text)
```

- Co vidíte v prohlížeči ≠ co stáhne program – část obsahu může doplňovat až JavaScript
- Automatizované stahování: respektujte `robots.txt` a pravidla webu; nestahujte opakovaně – uložte si kopii do souboru

Zpracování textu

- **Úroveň znaků** – např. filtrace a přemapování znaků, změna velikosti písmen
- **Úroveň slov** – tokenizace, počítání četností, slovník
- **Úroveň vět** – kontext: slova před a za aktuálním slovem, kontextové statistiky, náhrada n-gramu jiným m-gramem
  - **n-gram** – n-tice slov jdoucích po sobě
  - unigram = 1 slovo, bigram = 2 slova, trigram = 3 slova
- **Ostatní** – např. struktura dokumentu (odstavce, tabulky, HTML)

Bigramy věty „kůň úpěl ďábelské ódy“:
`(kůň, úpěl), (úpěl, ďábelské), (ďábelské, ódy)`

Metody řetězců

Vybrané metody řetězců (`str`):

- **Velikost písmen** – `lower()`, `upper()`, `capitalize()`, `title()`, `casefold()`
- **Ořezání a zarovnání** – `strip()`, `lstrip()`, `rstrip()`; `ljust(šířka)`, `rjust(šířka)`, `center(šířka)` (volitelně i výplňový znak)
- **Vyhledávání** – `count()`; `find()`, `rfind()` (nenalezeno → `-1`); `index()`, `rindex()` (nenalezeno → `ValueError`)
- **Dotazy** – `isalpha()`, `isdigit()`, `isalnum()`, `isspace()`, `islower()`, `isupper()`, `startswith()`, `endswith()`
- **Náhrada** – `replace(co, čím)`; hromadné přemapování znaků `translate()` + `str.maketrans()`
- **Dělení a spojování** – `split()`, `splitlines()`, `partition()`, `join()`

```
>>> 'Plzeň'.translate(str.maketrans('ň', 'n'))
'Plzen'
```

Úroveň znaků


```
text = 'z.n7a?ky@ z8ů12stanou'
znaky = []
for ch in text:
    if ch.isalpha() or ch.isspace():
        znaky.append(ch)
novy = ''.join(znaky)
print(novy)                 # znaky zůstanou
```

- `for ch in text:` prochází řetězec po znacích (řetězce délky 1)
- `isalpha()` funguje i pro českou diakritiku – metody `str` pracují s Unicode
- Skládání pomocí `''.join(seznam)` je efektivnější než opakované `novy += ch`
- Zdrojový soubor `.py` ukládejte v **UTF-8**; deklarace `# coding: cp1250` je přežitek

Úroveň slov


```
text = 'split rozdělí řetězec'
slova = text.split()
for idx, s in enumerate(slova):
    slova[idx] = 'ne' + s
text = ' '.join(slova)
print(text)         # nesplit nerozdělí neřetězec
```

- Rozdělení textu na slova (tokenizace):
  - `.split()` bez argumentu dělí podle libovolných bílých znaků – interpunkce ale zůstane přilepená ke slovům
  - regulární výraz, např. `re.findall(r'\w+', text)`
- Procházení seznamu – cyklus `for`
- Potřebujeme-li i index – `enumerate()` vrací dvojice `(index, prvek)`
- Kratší zápis: `' '.join('ne' + s for s in slova)`

Četnosti slov


```
import string
from collections import Counter
text = ("Příliš žluťoučký kůň úpěl ďábelské ódy. "
        "Příliš žluťoučký kůň běžel rychle.")
# 1. normalizace: malá písmena, interpunkce -> mezera
text_lower = text.lower()
for znak in string.punctuation + '„“”':
    text_lower = text_lower.replace(znak, ' ')
# 2. rozdělení na slova
slova = text_lower.split()
# 3. četnosti slov
frekvence = {}
for s in slova:
    frekvence[s] = frekvence.get(s, 0) + 1
print('Počet různých slov:', len(frekvence))
for slovo, pocet in frekvence.items():
    print(f'  {slovo}: {pocet}')
# totéž: Counter(slova).most_common()
```

Úroveň vět


```
import re
text = ("Příliš žluťoučký kůň úpěl ďábelské ódy. "
        "Příliš žluťoučký kůň běžel rychle.")
vety = [v.strip() for v in re.split(r'[.!?]+', text) if v.strip()]
pocty_slov = [len(v.split()) for v in vety]
prumer = sum(pocty_slov) / len(pocty_slov) if vety else 0
print('Počet vět:', len(vety))
print(f'Průměrný počet slov na větu: {prumer:.1f}')
```

Výstup:

```
Počet vět: 2
Průměrný počet slov na větu: 5.5
```

Pozor: dělení podle teček je naivní – selže na zkratkách, datech a desetinných číslech.

Otázka: Kolik vět najde tento kód v textu „Dne 7. 10. 2025 přednášel doc. Novák. Bylo to zajímavé.“?

Normalizace textu

Normalizace textu – převod do jednotné formy pro snazší zpracování:

- **Case-folding** – převod na malá písmena (`lower()`, důsledněji `casefold()`), aby „Velké“ a „velké“ bylo totéž slovo
- **Odstranění interpunkce a nežádoucích znaků** – čísla, symboly, emoji… podle úlohy
- **Unicode normalizace** – stejný znak může mít více zápisů: „č“ jako jeden kódový bod (NFC), nebo „c“ + kombinující háček (NFD) → před porovnáním sjednotit: `unicodedata.normalize('NFC', s)`
- **Odstranění diakritiky** – např. pro fulltextové vyhledávání (ř → r, ě → e):

```
import unicodedata
def bez_diakritiky(s):
    nfd = unicodedata.normalize('NFD', s)
    return ''.join(ch for ch in nfd if not unicodedata.combining(ch))
bez_diakritiky('Příliš žluťoučký')    # 'Prilis zlutoucky'
```

Normalizace a statistiky

- **Lemmatizace / stemming** – převod slova na základní tvar (lemma), resp. odstranění koncovek; vyžaduje lingvistické nástroje (zde jen zmínka)
- **Odstranění stop-slov** – velmi častých slov s malou obsahovou informací (cs: „a“, „v“, „na“, „je“; en: „the“, „a“, „to“)

Základní statistiky textu:

- počty znaků (celkem; písmena, číslice, mezery…)
- počty slov a vět, průměrná délka slova a věty
- nejčastější slova, rozložení délek slov, počet unikátních slov
- bohatost slovní zásoby – **type/token ratio**:

```
TTR = počet různých slov / počet všech slov
```

- četnosti n-gramů (nejčastější dvojice slov…)
- specifické počty – výskyty frází, podíl písmen a mezer apod.

Regulární výrazy

Regulární výraz (regular expression, regex, RE) je řetězec, který popisuje celou **množinu řetězců**. V teorii formálních jazyků popisuje **regulární jazyk**.

Využití:

- **vyhledávání** – zda text vyhovuje vzoru; kde v textu je shoda
- **manipulace s textem** – rozdělení e-mailové adresy na jméno a doménu; nahrazení částí textu (např. čísel jejich textovým zápisem); extrakce všech shod

Poznámky:

- Zavedl je americký matematik **Stephen Cole Kleene** (50. léta 20. století)
- Knihovny jako Python `re` umějí víc než teorie (např. zpětné reference `\1`) – popíší i některé **neregulární** jazyky
- Dokumentace: docs.python.org/3/library/re.html a Regular Expression HOWTO

RE – základy

Regulární výraz se skládá z **literálů** (znaky, které musí v textu být doslova) a **metaznaků** (znaky se zvláštním významem) – podobně jako `*` a `?` ve vzorech souborů (`*.txt`).

- `.` – právě jeden libovolný znak (kromě nového řádku)
- **kvantifikátory** – kolikrát se smí opakovat předchozí prvek
- `[ ]` – množina znaků: `[a-e]`, `[A-Z]`, `[1-5]`; negace `[^ab]` = vše kromě a, b
- `( )` – skupina: opakování sekvence, zachycení části shody
- `|` – alternativa; rozsah určují závorky: `telef(on|ax)`
- `^`, `$` – začátek a konec řetězce (s `re.MULTILINE` řádku)
- `\b` – hranice slova
- `\` – escapování a předdefinované třídy `\d`, `\w`, `\s` …

RE – kvantifikátory

- `?` – 0× nebo 1×
- `*` – 0× a více
- `+` – 1× a více
- `{n}` – přesně n×; `{m,n}` – m× až n×; `{m,}` – alespoň m×

Kvantifikátor se vztahuje k **předchozímu prvku** – znaku, třídě `[…]` nebo skupině `(…)`.

Kvantifikátory jsou **hladové** (greedy): zachytí co nejvíce znaků a ubírají jen tehdy, když by shoda jinak selhala.

Líné (lazy) varianty – přidáme `?`: `??`, `*?`, `+?`, `{m,n}?`, `{m,}?` – zachytí co nejméně znaků.

```
re.findall(r'<b>.*</b>',  '<b>a</b> <b>c</b>')
# ['<b>a</b> <b>c</b>']
re.findall(r'<b>.*?</b>', '<b>a</b> <b>c</b>')
# ['<b>a</b>', '<b>c</b>']
```

RE – třídy znaků

- `\d` – číslice; `\D` – cokoli kromě číslic
- `\w` – znaky „slova“: písmena, číslice, _; `\W` – doplněk
- `\s` – bílé znaky (mezera, tabulátor, konce řádků); `\S` – doplněk

Pozor – v Pythonu 3 jsou třídy unicodové:

- `\w` zahrnuje i `č, ř, ž, ö, ü…`; ekvivalent `[a-zA-Z0-9_]` platí jen s příznakem `re.ASCII`
- `\d` zahrnuje i číslice jiných písem (např. arabsko-indické ٣); jen 0–9 → `[0-9]` nebo `re.ASCII`

```
re.findall(r'\w+', 'Žluťoučký kůň')
# ['Žluťoučký', 'kůň']
re.findall(r'\w+', 'Žluťoučký kůň', re.ASCII)
# ['lu', 'ou', 'k', 'k']
```

RE – escapování

Metaznaky: `\ ^ $ . [ ] | ( ) ? * + { }`

Chceme-li je použít v původním významu, **escapujeme** je zpětným lomítkem `\`.

Příklad – vyhledání rovnice `2*(a+b) = c`:
`2\*\(a\+b\) = c`

- Uvnitř `[ ]` většina metaznaků ztrácí zvláštní význam: `[.*+]` = tečka, hvězdička nebo plus
- Automatické escapování: `re.escape('2*(a+b) = c')`
- Vzory pište jako **raw řetězce** `r'...'` – jinak Python sám interpretuje `\` (`'\b'` = backspace!)

RE – příklady

- `a+` – sekvence písmen a (1 a více)
- `a*` – sekvence písmen a (0 a více)
- `o?kov` – okov či kov
- `tel(efon)?` – tel či telefon
- `telef(on|ax)` – telefon či telefax
- `^(?:[0-9]|[1-9][0-9])$` – celý řetězec je číslo 0 až 99 (bez úvodní nuly)
- `\d{2}` – dvě číslice (00, 01, …, 99)
- `[0-9a-fA-F]+` – posloupnost hexadecimálních číslic
- `\b(?:19|20)\d{2}\b` – letopočty 1900–2099 jako samostatné slovo
- `\d{2,6}` – dvě až šest číslic
- `[^ ,.]+` – neprázdná sekvence znaků bez mezery, čárky a tečky

Proč ukotvení? `re.search(r'[0-9]|[1-9][0-9]', '42')` najde jen `'4'` – vyhraje první vyhovující alternativa na nejlevější pozici.

RE – příklady

- `^P.*` – řetězec začínající písmenem P, za nímž následuje libovolný (i nulový) počet znaků
- `\d+0$` – řetězec končící nulou, které předchází alespoň jedna číslice
- `a+b` – ab, aab, aaab, …
- `a\+b` – doslova a+b

Zjednodušený vzor e-mailové adresy:
`^[_a-zA-Z0-9-]+(\.[_a-zA-Z0-9-]+)*@[a-zA-Z0-9-]+(\.[a-zA-Z0-9-]+)*$`

(Úplná validace e-mailu regexem je nepraktická; adresa se ověřuje odesláním e-mailu.)

Další konstrukce v Pythonu:

- `(?aimsx)` – příznaky přímo ve vzoru: ASCII, ignore case, multiline, dot matches all, verbose
- `(?:…)` – skupina bez zachytávání
- více: docs.python.org/3/library/re.html

Modul re – funkce

Modul `re` – nejdůležitější funkce:

- `re.search(pattern, text)` – první výskyt **kdekoliv** v textu → match objekt, nebo `None`
- `re.match(pattern, text)` – shoda **na začátku** textu (jako by vzor začínal ^). Nezaměňovat s match objektem, který je výsledkem.
- `re.fullmatch(pattern, text)` – shoda s **celým** textem (kontrola formátu)
- `re.findall(pattern, text)` – **všechny nepřekrývající se** výskyty → seznam řetězců (nebo n-tic, má-li vzor skupiny)

```
re.match(r'\d+', 'a123')        # None
re.search(r'\d+', 'a123')       # <re.Match ...; match='123'>
re.fullmatch(r'\d+', '123a')    # None
```

Modul re – další funkce

- `re.finditer(pattern, text)` – jako `findall`, ale vrací iterátor match objektů (pozice nálezů, paměťově úsporné)
- `re.sub(pattern, repl, text, count=0)` – nahradí výskyty řetězcem `repl` (i prázdným = odstranění); v `repl` lze odkazovat na skupiny `r'\1'`; `repl` může být i **funkce**
- `re.split(pattern, text)` – rozdělení podle vzoru (jako `str.split`, ale se vzorem místo pevného oddělovače)
- `re.compile(pattern)` – předkompilovaný vzor pro opakované použití

```
datum = re.compile(r'(\d{1,2})\. ?(\d{1,2})\. ?(\d{4})')
datum.sub(r'\3-\2-\1', 'Termín: 7. 10. 2025')
# 'Termín: 2025-10-7'
```

RE – match objekt

Match objekt vracejí `search`, `match`, `fullmatch` a `finditer` při nalezení shody:

- `m.group()` nebo `m.group(0)` – celý nalezený podřetězec
- `m.group(1)`, `m.group(2)`, … – obsah zachycených skupin
- `m.group('jmeno')` – pojmenovaná skupina `(?P<jmeno>…)`
- `m.groups()` – n-tice všech skupin; `m.groupdict()` – slovník pojmenovaných skupin
- `m.start()`, `m.end()` – index začátku a konce shody
- `m.span()` – dvojice `(start, end)`

Funkce vracejí `None`, pokud shodu nenajdou → výsledek vždy testujte:

```
m = re.search(vzor, text)
if m:
    print(m.group())
```

RE – skupiny

Vyhledání řetězce a získání skupin:

```
import re
m = re.search(r'([a-zA-Z0-9_.]+)@([a-zA-Z0-9_.]+)',
              'Moje adresa je smidl@kky.zcu.cz, děkuji.')
print(m.group(0))   # smidl@kky.zcu.cz
print(m.group(1))   # smidl
print(m.group(2))   # kky.zcu.cz
```

RE – pojmenované skupiny

Vyhledání řetězce a získání **pojmenovaných skupin**:

```
import re
m = re.search(r'(?P<jmeno>[\w.]+)@(?P<host>[\w.]+)',
              'Moje adresa je smidl@kky.zcu.cz, děkuji.')
print(m.group(0))           # smidl@kky.zcu.cz
print(m.group('jmeno'))     # smidl
print(m.group('host'))      # kky.zcu.cz
print(m.groupdict())
# {'jmeno': 'smidl', 'host': 'kky.zcu.cz'}
```

RE – všechny výskyty

Získání všech e-mailových adres:

```
import re
text = ('Zašlete na info@seznam.cz, '
        'honzas@kky.zcu.cz, smidl@gmail.com')
adresy = re.findall(r'[\w.]+@[\w.]+', text)
print(adresy)     # seznam řetězců
dvojice = re.findall(r'([\w.]+)@([\w.]+)', text)
print(dvojice)    # seznam n-tic se skupinami
for m in re.finditer(r'([\w.]+)@([\w.]+)', text):
    print(m.group(0), m.span())
```

Otázka: Co vrátí `re.findall(r'[\w.]+@[\w.]+', 'Pište na info@seznam.cz.')`?

RE – anonymizace


```
import re
text = ('Zašlete na info@seznam.cz, '
        'honzas@kky.zcu.cz, smidl@gmail.com')
email = r'([\w.]+)@([\w.]+)'
print(re.sub(email, 'EMAIL', text))
print(re.sub(email, r'anonym@\2', text))
def maskuj(m):
    return m.group(1)[0] + '***@' + m.group(2)
print(re.sub(email, maskuj, text))
```

Výstup:

```
Zašlete na EMAIL, EMAIL, EMAIL
Zašlete na anonym@seznam.cz, anonym@kky.zcu.cz,
           anonym@gmail.com
Zašlete na i***@seznam.cz, h***@kky.zcu.cz, s***@gmail.com
```

RE – rozdělení textu

Řezání podle regulárního výrazu:

```
import re
data = 'Why    is this  \t \t  wrong'
a = re.split(' ', data)
b = re.split(r'\s+', data)
c = data.split()
print(a)
# ['Why', '', '', '', 'is', 'this', '', '\t', '\t', '', 'wrong']
print(b)   # ['Why', 'is', 'this', 'wrong']
print(c)   # ['Why', 'is', 'this', 'wrong']
```

`str.split()` bez argumentu také dělí podle libovolných bílých znaků;
`re.split` potřebujeme, když oddělovač není pevný, např. `r'[,;]\s*'`.

RE – pokročilé techniky

Pro úplnost:

- **Lookahead / lookbehind** (dopředné / zpětné vyhlížení) – podmínka na to, co **následuje** `(?=…)` / **nesmí následovat** `(?!…)`, resp. co **předchází** `(?<=…)` / **nesmí předcházet** `(?<!…)`; do shody se nezahrne

```
re.findall(r'\d+(?= Kč)', 'Cena: 300 Kč, sleva 20 %')
# ['300']
```

- **Skupiny bez zachytávání** `(?:…)` – jen logické seskupení, skupina se nečísluje; přehlednější výstup `findall` i `groups()`

```
re.findall(r'(?:ab)+', 'ababx abx')
# ['abab', 'ab']
```

RE – příznaky

- `re.IGNORECASE` / `(?i)` – nerozlišuje velká a malá písmena
- `re.MULTILINE` / `(?m)` – `^` a `$` platí pro začátek a konec každého řádku
- `re.DOTALL` / `(?s)` – tečka `.` zahrnuje i znak nového řádku
- `re.VERBOSE` / `(?x)` – vzor s mezerami a komentáři `#` pro čitelnost
- `re.ASCII` / `(?a)` – `\w`, `\d`, `\s`, `\b` jen pro znaky ASCII

Kombinace příznaků operátorem `|`:

```
re.search(pattern, text, flags=re.IGNORECASE | re.MULTILINE)
telefon = re.compile(r'''
    (?:\+420\s?)?            # nepovinná předvolba
    \d{3}\s?\d{3}\s?\d{3}    # 9 číslic, volitelně po trojicích
''', re.VERBOSE)
```

Shrnutí a otázky

- **Soubory:** vždy `with` a vždy `encoding`; binární data v režimu `'b'`
- **Kódování:** `str` ↔ `bytes` přes `encode()`/`decode()`; preferujte UTF-8; mojibake = špatné kódování při dekódování
- **Text:** normalizace (malá písmena, interpunkce, NFC), tokenizace, statistiky
- **RE:** metaznaky, kvantifikátory (greedy/lazy), unicodové `\w`, ukotvení `^ $ \b`, `search/match/fullmatch/findall/sub/split`

Otázky k zamyšlení:

- Proč může stejný program načíst stejný soubor na Windows a na Linuxu různě?
- Kolik bajtů zabere v UTF-8 řetězec `'kůň'`?
- Napište regex, který najde česká PSČ ve tvaru `301 00` i `30100`.
- Kdy je lepší místo regulárního výrazu použít parser (např. pro HTML)?

Cvičení: stažení seznamu zaměstnanců KKY z webu, zpracování a uložení do souboru.